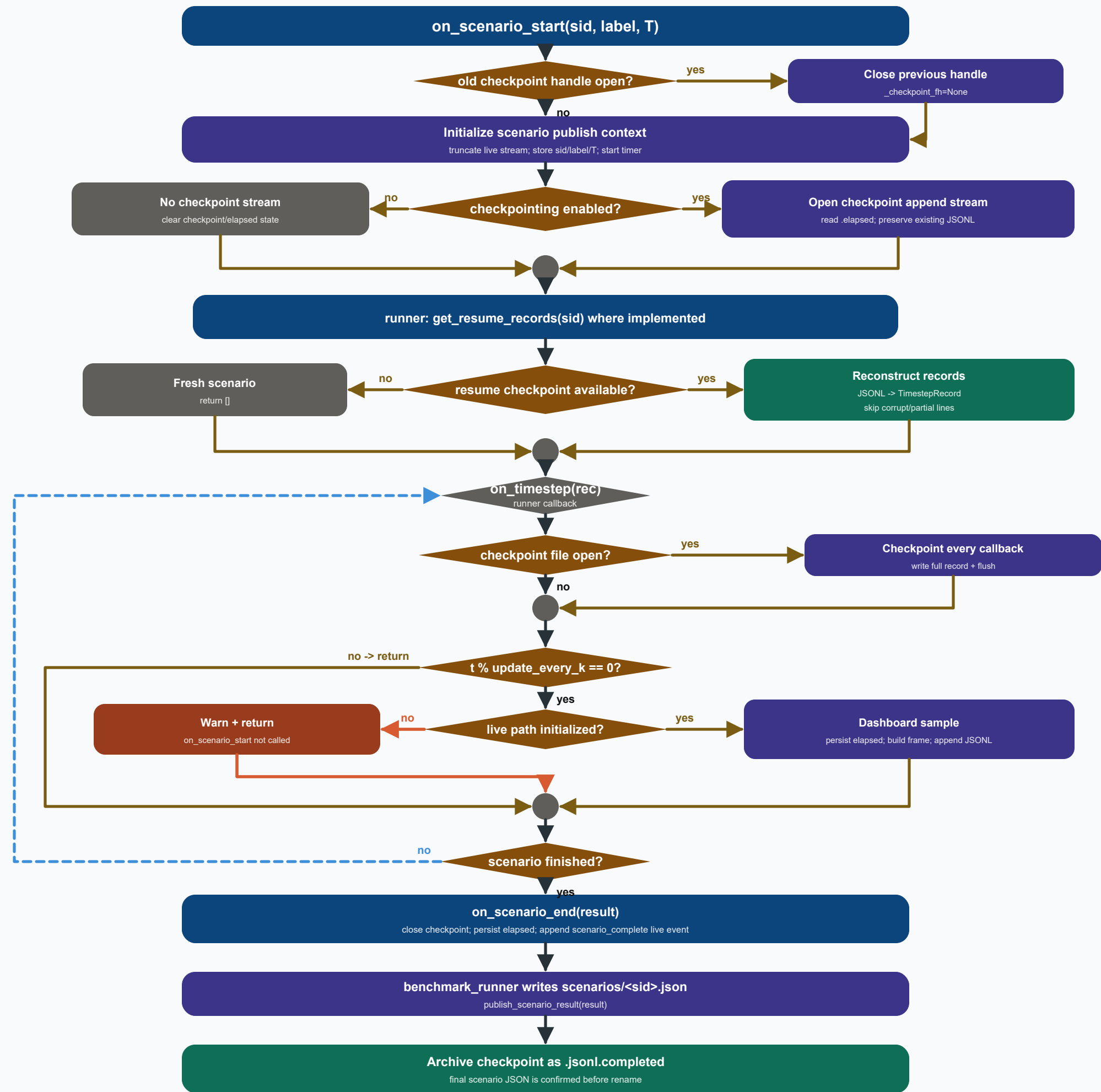




Two independent JSONL streams

`live/<sid>.jsonl`: dashboard-only compact frames; always truncated at `on_scenario_start()`.

`checkpoint/<sid>.jsonl`: full `TimestepRecord` serialization; opened in append mode and never truncated by `PublishHandle`.

`update_every_k` affects only live dashboard frames. Checkpoint writes occur on every `on_timestep()` callback.

`.elapsed` stores cumulative wall-clock time across crash/restart attempts.

`cumulative_elapsed_s()` = prior persisted elapsed + current attempt runtime.

Resume and completion semantics

`get_resume_records()` first checks `checkpoint/<sid>.jsonl`, then `.jsonl.completed` if the live path is absent.

Valid JSONL rows are reconstructed with `TimestepRecord.from_checkpoint_dict()`.

Corrupt or partial lines are skipped, which protects against a crash during the final append.

After a successful runner return, `benchmark_runner` writes the final scenario JSON and then renames the checkpoint to `.completed`.

Layer 1 resume in `benchmark_runner` can skip the runner entirely when `scenarios/<sid>.json` already exists and loads correctly.

Runner-integration caveats

`PublishHandle` can only checkpoint records that a scenario runner actually passes to `on_timestep()`.

S2/S3 pre-PF failure branches append failed records directly and therefore skip publisher `checkpoint/live` callbacks for those steps.

S1 accepts the publisher handle but does not call `get_resume_records()`, so crash-resume is not implemented for its `run_timeseries` sweep.

S2-S5 call `on_scenario_start()` before `get_resume_records()`; partial resumes continue from the last recovered record.

`on_scenario_end()` closes the open checkpoint handle before appending the terminal live event.